



דוח מחקר

אבטחת המרחב המקוון

מגישה: יפית ברנר

Yafit.bre@gmail.com



תוכן עניינים:

1	שער
2	תוכן עניינים
3	מבוא
6	מתודולוגיה
9	תוצאות, ניתוח ומסקנות
11	גרפים
15	שיקולים אתיים
16	מקורות

מבוא

דוח זה מציג ניתוח השוואתי של מנגנוני אימות וסיסמאות, אשר בוצע בסביבת שרת וירטואלית המדמה ממשק REST API מקומי. מטרת הניסוי המרכזית היא להמחיש בצורה אמפירית את הפער הדרמטי בין פונקציות גיבוב מהירות יחסית למשל SHA256 לבין פונקציות Memory Hard המיועדות להאט תוקפים. המחקר בוחן את עמידותם של אלגוריתמי הגיבוב Argon2id, SHA256, Bcrypt אל מול מתקפות סייבר נפוצות מסוג Brute Force וגם Password Spraying במסגרת הניסוי נבדקה היעילות של שכבות הגנה שונות:

מנגנוני חסימה:

שימוש ב Rate Limiting למניעת הצפות וויסות עומסים, ובאמצעות Account Lockout לעצירת תוקף סדרתי המנחש סיסמאות, מנגנון שהוכח כגורם החסימה היעיל ביותר בניסוי. הוספת Pepper שהוא רכיב סוד גלובלי המופרד מבסיס הנתונים ומחזק את שמירת הסיסמאות מפני דליפה.

אימות מתקדם:

מניעת אוטומציה ע"י דרישת CAPTCHA ואימות דו-שלבי TOTP הכולל מנגנון סנכרון לתיקון סטיות שעון Drift

המחקר אוסף איסוף מדדים כמותיים מדויקים בהם מספר ניסיונות לשנייה, זמן לפריצה ושיעורי הצלחה לפריצה. לבסוף, השווינו בין שרת לא מוגן לבין שרת עם הגנה לעומק כדי למדוד את ההשפעה המצטברת על מהירות הפריצה ועל אחוזי ההצלחה של התוקף. עקרונות היסוד באבטחת מנגנוני אימות, האיומים המרכזיים עמם מתמודד הפרויקט, ואת הטכנולוגיות שנבחרו למימוש ההגנה:

Authentication

תהליך האימות הוא השלב בו המערכת מאמתת את זהות המשתמש שיש שקיימת לו גישה למערכת. השיטה בפרויקט הזה היא שיטה של אימות זהות של משתמש וסיסמא, וגם הפרויקט בודק אימות של נתון קיים TOTP.

Hashing

אלגוריתמים של שמירת סיסמאות מסוג Plain Text היא כשל אבטחה חמור, למשל במקרה של דליפת בסיס הנתונים, התוקף יקבל גישה מיידית לכל החשבונות לכן הפתרון הסטנדרטי לבעיה זו הוא שימוש בפונקציות גיבוב Hashing. תהליך הגיבוב הוא פונקציה מתמטית חד-כיוונית הממירה את הסיסמה המקורית לרצף תווים ייחודי אבל בניגוד להצפנה רגילה, פעולה זו אינה הפיכה, כך שלא ניתן לשחזר את הסיסמה המקורית מתוך הערך המגובב שנשמר בבסיס הנתונים.

במהלך המחקר נבדקו שלושה אלגוריתמים המייצגים דורות שונים של אלגוריתמים:

SHA 256

פונקציית גיבוב קריפטוגרפית מהירה מאוד ממשפחת SHA2 היא תוכנה במקור להיות יעילה חישובית לצרכי חתימות דיגיטליות, תכונה המהווה חיסרון מובהק באבטחת סיסמאות כיוון שהיא מאפשרת לתוקף לבצע מיליוני ניחושים בשנייה בחומרה סטנדרטית. בפרויקט זה האלגוריתם מומש בצורה בסיסית עם Salt כדי להמחיש את חולשתו מול תוקף מהיר.

Bcrypt

אלגוריתם ייעודי לשמירת סיסמאות המבוסס על צופן Blowfish הוא כולל מנגנון שמאפשר להגדיר Work Factor זהו פרמטר המאט את פעולת הגיבוב במכוון האטה זו הופכת את תהליך הניחוש לארוך יותר עבור התוקף, ומאפשרת להגנות להתאים את הקושי ככל שהחומרה מתחזקת.

Argon2id

אלגוריתם לגיבוב סיסמאות שנחשב כיום לסטנדרט בתחום אבטח המידע. זהו אלגוריתם שדורש לא רק זמן מעבד אלא גם הקצאת זיכרון משמעותית לכל חישוב. בכך מנטרל את היתרון של תוקפים המשתמשים בחומרה ייעודית GPU ומספק הגנה משולבת גם מפני התקפות. על מנת להעריך את יעילותן של שיטות ההגנה האלה אז צריך להבין מהם האיומים שצריך להתמודד ולממש בתהליך המחקר, הפרויקט מבצע הדגמה של תקיפות נפוצות:

Brute Force Attack

מתקפה המכוונת כנגד משתמש יחיד, בה התוקף מנסה בצורה שיטתית רצף של צירופים עד למציאת הסיסמה או עד לנעילת החשבון. יעילות התקיפה תלויה ישירות בכוח המחשוב של התוקף ובמהירות התגובה של השרת, כלומר מהירות ה Latency

Dictionary Attack

גרסה יעילה יותר של Brute Force בה התוקף משתמש ברשימה מוכנה מראש של מילים נפוצות וסיסמאות שדלפו (למשל קובץ rockyou.txt) וההנחה היא שמשתמשים לפעמים עלולים לבחור סיסמאות בעלות משמעות ולא רצפים אקראיים.

Password Spraying

בניגוד ל Brute Force המתמקד במשתמש אחד, תקיפה זו היא רחבה יותר, התוקף מנסה מספר מצומצם של סיסמאות נפוצות מאוד עם רשימה ארוכה של שמות משתמשים. מטרת השיטה היא לעקוף מנגנוני הגנה מסוג נעילת חשבון המופעלים בדרך כלל רק לאחר מספר כישלונות ברצף לכניסה ואז להישאר מתחת לדאאר של מערכות ההגנה.

מנגנוני הגנה:

בפרויקט נבדקו שכבות הגנה שונות שנועדו לפצות על חולשות בסימאות המשתמשים:

Salt

ערך רנדומלי וייחודי הנוצר לכל משתמש ונשמר בצמוד לסימא ב DB לפני הגיבוב ומטרתו למנוע שימוש בטבלאות ידועות מראש וגם לדאוג שלא שבמקרה שיש 2 משתמשים עם אותה סימא אז שיקבלו גיבוב שונה כל אחד מהם.

Pepper

רכיב סודי שמתווסף לסימא לפני הגיבוב.
אבל בשונה מה Salt הוא נשמר בנפרד מבסיס הנתונים בפרויקט על מנת לספק הגנה גם במקרה של דליפת כל ה DB.

Lockout & Rate Limiting

מנגנוני חסימה להגבלת בקשות הכניסה לחשבון המשתמש.
Rate Limiting מבצע חסימה ברמת השרת על מנת למנוע הצפות לעומת Lockout שהוא בעצם נועל את החשבון לגמרי לאחר מספר מוגדר של כישלונות רצופים וכך חוסם את תקיפת Force Brute.

TOTP

הוא פרוטוקול המוגדר בתקן RFC 6238 הוא מנגנון אימות דו-שלבי (MFA) האלגוריתם מייצר סימא דינמית חד-פעמית המשתנה כל 30 שניות, המחושבת על בסיס סוד משותף והזמן הנוכחי בשרת ובצד הלקוח.
השיטה הזאת מספקת הגנה חזקה מפני גניבת סימאות סטטיות מפני שהקוד תקף לזמן קצר בלבד אבל התלות בזמן מחייבת התמודדות עם Clock Drift בין הצדדים. בפרויקט זה מומש מנגנון סנכרון המאפשר לשרת לזהות סטייה משמעותית, לחשב את ההיסט Offset ולבצע תיקון כדי לאפשר אימות תקין מבלי לפגוע ברמת האבטחה.

Captcha

בדיקה ויזואלית שנועדה להבדיל בין משתמש אמיתי לבוט אבל בפרויקט מנגנון זה ממומש כסימולציה לוגית ולא כבדיקה ויזואלית אמיתית. השרת עוקב אחרי מספר הניסיונות הנכשלים של המשתמש וכאשר הוא עובר רף מסוים אז השרת משנה את הסטטוס של המשתמש ומחזיר רק ערך בוליאני ואז כל בקשת התחברות נוספת ללא טוקן נדחית אוטומטית אפילו אם היא נכונה.
לכן הפרמטר הסודי שמומש בפרויקט הוא group_seed שמשמש רק לבדיקות ולא מממש תקיפה אמיתית

מתודולוגיה

המערכת פותחה בשפת פייתון ומומשה בארכיטקטורת שרת לקוח מקומית המדמה תקשורת מול שרת REST API בסביבה מבודדת ומבוקרת.

שרת האימות - Auth_server.py

זהו צד שרת המומש בעזרת ספריית Flask שמנהל את ה DB המנוהל בקבצי JSON. הוא אוסף את מדיניות האבטחה הנדרשות לפרויקט (Rate Limiting, Lockout) ומבצע את פעולות הגיבוב. טיפול באימות דו-שלבי : TOTP שימוש בספריית Pyotp לאימות הטוקן מול ה-Secret של המשתמש וניהול סנכרון שעונים Drift זמן התגובה נמדד בצד השרת , מרגע קליטת הבקשה ועד סיום הטיפול בה ונשמר ב Latency_ms נבחרה שיטה זו כדי לנטרל רעשי רשת ולמדוד נטו את זמן העיבוד של האלגוריתמים ומנגנוני ההגנה. השרת מנהל מונה לניסיונות כושלים עבור כל משתמש וברגע שהערך שלו עובר את ההגדרה בקובץ config המתאים לניסוי המסוים אז השרת מונע גישה למשתמש/תוקף.

התוקף attacker.py

סקריפט שמבצע בקשות שוב ושוב לנקודות הקצה של Login & Login_TOTP והוא בעצם מדמה Brute Force & Password Spraying סקריפט התקיפה הוא כמו סוג של בוט חכם המגיב בזמן אמת לתשובות השרת. הלוגיקה מבוססת על ניתוח קודי תגובה ופרמטרים בגוף תשובת קבצי ה JSON

users.json

users1-10 weak :סימאות נפוצות וחלשות שנלקחו מתוך מאגר הדליפות הידוע rockyou.txt שממומש בפרויקט ב wordlist.txt
users 11-20 medium :משתמשים עם סימאות באורך 8-10 תווים שמדמות משתמש ממוצע.
Users 21-30 strong :סימאות רנדומליות נפוצות עם אימות דו שלבי פעיל TOTP

main.py

הוא בעצם נקודת הכניסה של המערכת ומנהל את כל הניסוי ואחראי על הסימולציות בצורה אוטומטית באמצעות הרצת כל קבצי הקונפיגורציה config_*.json עבור כל ניסוי הוא מריץ את השרת והתוקף כסוג של תהליכים נפרדים ומנהל אותם בסיום כולל בדיקה שכל הלוגים עברו לתיקייה המתאימה לאחר ההרצה.

logic.py

הוא מודל ניתוח הנתונים שאחראי על ניתוח הנתונים לאחר סיום כל הניסויים. משתמש בספריית Pandas על מנת לטעון ת קבצי ה JSON וגם מבצע סטטיסטיקות לחישוב מדדים כמו אחוז הצלחה לפי סוג משתמש או זמן תגובה, ממוצע או סוגי הגנות או סוגי תקיפות. בנוסף משתמש בספריית Matplotlib על מנת לייצר את קבצי הגרפים.

[README.md](#)

תיעוד טכני של הפרויקט, הוראות התקנה, מבנה תיקיות והנחיות להרצה.

[create_data.py](#)

הוא סקריפט ההתקנה של הניסוי והוא אחראי על יצירת קובץ ה- `users.json` ומשתמש ב `group_seed` על מנת לייצר את המשתמשים, הסיסמאות וגם ה `Secrets` ל `TOTP` בצורה רנדומלית. בנוסף הוא מחשב את ה `Hash` למשתמשים ב `DB` לפני הרצת השרת.

[run_experiments.py](#)

הוא הסקריפט שמנהל את לולאת הניסויים באמצעות קריאת כל קבצי הקונפיגורציה `config_*.json` ועבור כל קובץ הוא מפעיל תהליך נפרד של השרת ושל התוקף. בנוסף הוא מנטר את סיום הריצה של התוקף ומבצע סגירה מסודרת של השרת וסידור הלוגים בתיקייה לפי תאריך ושעת הריצה.

[report_gen.py](#)

הוא מפיק את הדוח לאחר שכל עיבוד הנתונים מסתיים ע"י `logic.py` בתהליך זה הוא אוסף את כל הפלטים של הפרויקט שהם הגרפים והסטטיסטיקות ומייצר קובץ דוח `HTML` שמציג את כל הניסוי.

[test_env.py](#)

הוא סקריפט של בדיקת סביבת הבדיקות לפני כל הניסויים והוא מוודא שכל הספריות מותקנות לפי הדרישות בקובץ `requirements.txt` וגם שתיקיות הפלט `logs & graphs` קיימות ויש להן הרשאת כתיבה. הוא בעצם מונע כישלון של הניסוי באמצע הריצה שעלול להיות בגלל בעיות בסביבה.

[run_all.bat](#)

סקריפט הרצה שמאפשר הרצה של כל הפרויקט בלחיצה אחת. וגם הוא מגדיר את ה `python path`

[run_experimant.ps1](#)

סקריפט הרצה שמאפשר להריץ כל פעם ניסוי בנפרד.

[requirments.txt](#)

קובץ שמכיל רשימה של ספריות הפיתוח החיצוניות הנדרשות להרצת הפרויקט.

[config *.json](#)

אוסף קבצי הגדרות ב `JSON` כאשר כל קובץ מייצג תרחיש ניסוי בודד ומבודד משאר הניסויים. חלוקה זאת לקבצים מאפשרת להריץ את הניסויים ולוודא שרק הפרמטרים הרצויים מתעדכנים בין הרצה להרצה. באמצעות שימוש בקבצים האלה, ה `main.py` יכול לטעון אותם ואז לאתחל את השרת עם ההגנות המתאימות לכל ניסוי ללא שינוי ידני בזמן ריצה.

- קבצי הקונפיגורציה שולטים על הפרמטרים בשרת בזמן אמת:
1. אלגוריתם הגיבוב שמחליט איפה הגנה תופעל (Sha256, Bcrypt, Argon2id)
 2. ערכים בוליאניים למנגנוני ההגנה:
(rate_limit_enabled, account_loockout_enabled)
 3. ערכים בוליאניים למנגנוני האימות: (captcha_enabled, totp_enabled_global)
 4. פרמטרים חישוביים: argon_mempry_cost, אורך ה SALT
שמאפשרים מעט שליטה על עומס העיבוד.

[create_wordlist.py](#)

סקריפט עזר המייצר את קובץ הסיסמאות הנפוצות.
וגם להרצה של תקיפה מסוג password spraying

[run_experiments.py](#)

סקריפט להרצת אלגוריתמי הגיבוב ומבצע השוואות ביניהם.
וגם מנהל את מנגנוני רישום וסנכרון של TOTP

[run_combinations.py](#)

סקריפט להרצת ניסויים המפעילים בכל פעם מנגנון הגנה יחיד כדי למדוד בנפרד את
האפקטיביות של כל הגנה בנפרד.

תוצאות, ניתוח ומסקנות:

הפרויקט מתבסס על הרצה שיטתית של וקטורי תקיפה מגוונים מול קונפיגורציות הגנה משתנות.

הפלט כולל קובצי לוג מפורטים המדמים תרחישי אמת, וניתוח ויזואלי של הנתונים באמצעות גרפים. מהניתוח עולה שקיים פער משמעותי ביכולת התוקף להשיג גישה למערכת עם מנגנוני ההגנה הפעילים.

ניתוח תקיפות ומנגנוני חסימה:

בניסוי ה-Spraying זוהתה מתקפה של מאות ניסיונות בזמן קצר. למרות שהמספר נמוך ממתקפות ענק, הוא מראה כיצד תוקף עוקף את מנגנון הנעילה האישי באמצעות דילוג בין משתמשים.

זמן התגובה הממוצע עמד על 0.0ms נתון זה ממחיש את הסכנה הקריטית בשימוש באלגוריתם גיבוב מהיר Sha256 ללא שכבות הגנה נוספות המאפשר לתוקף לסרוק מילונים ענקיים בשניות בודדות.

לעומת זאת, בתרחיש של הגנה מלאה sha256_full_protection.log הכולל Lockout & Rate Limit מספר הניסיונות של התוקף נבלם אחרי 5 ניסיונות בלבד ואז החשבון ננעל והתקיפה נוטלה לחלוטין. במנגנון Rate Limit הייתה השהייה של שנייה אז הוא היעיל ביותר בעצירת מתקפות אוטומטיות.

השוואת ביצועים ועלות גיבוב:

קיימת העדפה בין אבטחה לביצועים:

SHA256 הציג את זמני התגובה הארוכים ביותר, עלות החישוב הגבוהה הופכת את תקיפת Brute Force לבלתי מעשית מבחינת זמן עבור התוקף אבל זמן תגובה זה עלול ליצור צוואר בקבוק ולפגוע בחווית המשתמש בעומסים גבוהים.

Bcrypt רץ בזמן עיבוד ממוצע של 164ms זאת האטה משמעותית המקשה על Brute Force אבל קלה יותר לעיבוד לשרת מאשר התוצאות הישנות. Argon2id רץ בזמן אופטימלי של 45ms המאזן בין אבטחה לחוויית משתמש. אז התוצאה של Argon2id היא הכי אופטימלית בגלל שנוצרת השהיה מספקת כדי להקשות על תוקפים בנוסף לשימוש בזיכרון המקשה על חומרה ייעודים אבל ללא פגיעה מורגשת בביצועי השרת.

לכן המסקנה היא ש Argon2id הוא בעל ההגנה היעילה ביותר בשקלול ביצועים לעומת SHA256 שממש פסול לשימוש בסיסמאות.

אפקטיביות הגנה על סיסמאות חלשות:

בניסוי עם משתמשים עם סיסמאות חלשות ונפוצות נפרצו כמעט ב 100% מהמקרים גם כאשר מנגנון Rate Limit היה פעיל ולא הופעלה הגנה, לפעמים התוקף ניחש את הסיסמא בניסיון הראשון או השני.

המסקנה היא שמנגנוני הבלימה מגיבים לפעמים לאט מדי כדי להגן מפני ניחוש ראשוני מוצלח וזה מוכיח את הצורה בשילוב של הגנות ולא ניתן להסתמך רק על מנגנון יחיד וחובה לאכוף מדיניות של סיסמאות חזקות.

TOTP וסנכרון זמנים:

בכל הניסויים הופעל מנגנון Drift, השרת זיהה בהצלחה סטייה של 90ms בשעון הלקוח וביצע סנכרון, זאת יכולת קריטית למניעת חסימת משתמשים אמיתיים (לא תוקפים) בגלל בעיות רשת או שעון תוך שמירה על רמת אבטחה גבוהה.

ניתוח סיבות הכישלון:

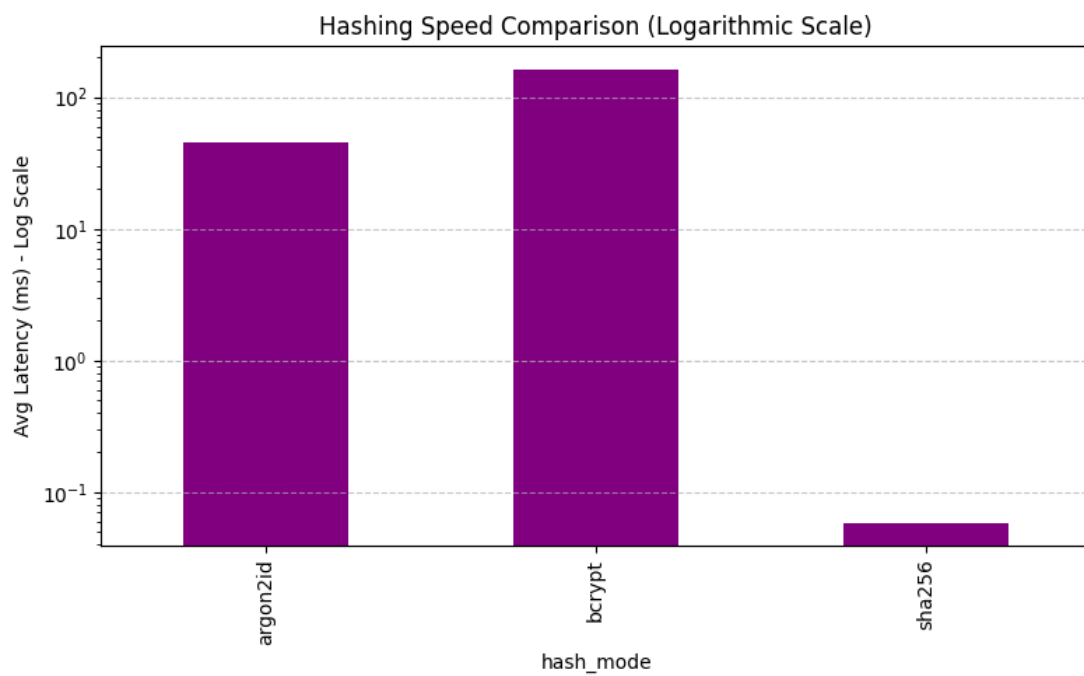
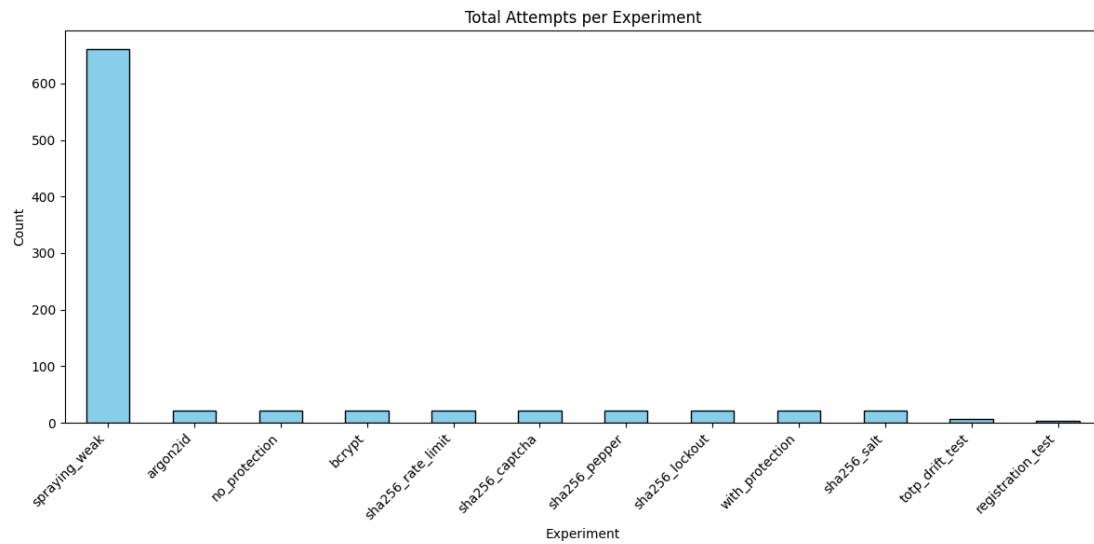
ניתוח סיבות הכישלון מראה כי 93% מניסיונות התקיפה נכשלו בגלל נעילת חשבון, זה מוכיח שמנגנון הנעילה, Account Lockout, הוא קו ההגנה האפקטיבי ביותר בפרויקט בגלל שהוא עוצר את התוקף לפני שנוספת למערכת פעולות גיבוב נוספות.

בגרף העמודה של Spraying_Weak מציגה מעל ל 35000 ניסיונות תקיפה שלא נחסמו מיד. מנגנון נעילת המשתמש מגנה מפני ניסיון תקיפה של הרבה סיסמאות על משתמש יחיד, לעומת בתקיפה רחבה של spraying התורף מנסה סיסמא אחת מסוימת על הרבה משתמשים שונים. לכן המסקנה היא שמנגנון הנעילה ברמת המשתמש היחיד אינו מספיק והמערכת צריכה להיות עם שכבת הגנה רחבה יותר בגלל שהתוקף הצליח לעקוף את המנגנון ובעצם להחליף משתמשים ולבצע אלפי ניסיונות מבלי שתהיה עליו חסימה.

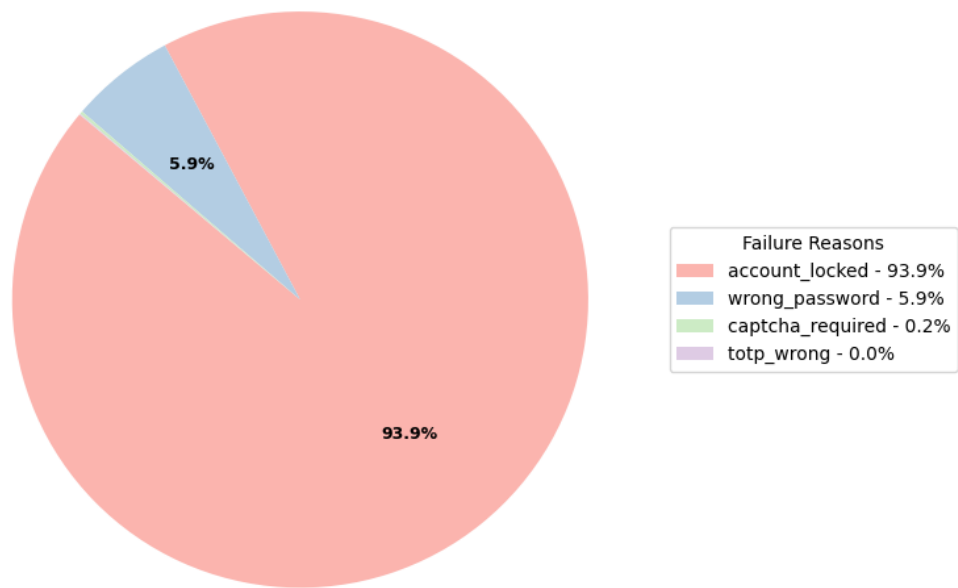
CAPTCHA

Bcrypt 164ms בכל ניסיון התחברות כלומר השרת משקיע משאבי מעבד משמעותיים בכל בקשה אז המסקנה היא שגיבוב איטי מגן מפני פריצת סיסמאות אבל הוא חושף את השרת למתקפות DOS, כלומר התוקף יכול להציף את השרת בבקשות התחברות ואז לגרום לשרת להיתקע כשיש חישובי גיבוב רבים ואז להפיל את המערכת למשתמשים אמיתיים (לא תוקפים) שבמקרה מנסים להתחבר. הפתרון הוא CAPTCHA אבל לפני שלב הגיבוב כדי למנוע את העומס הזה.

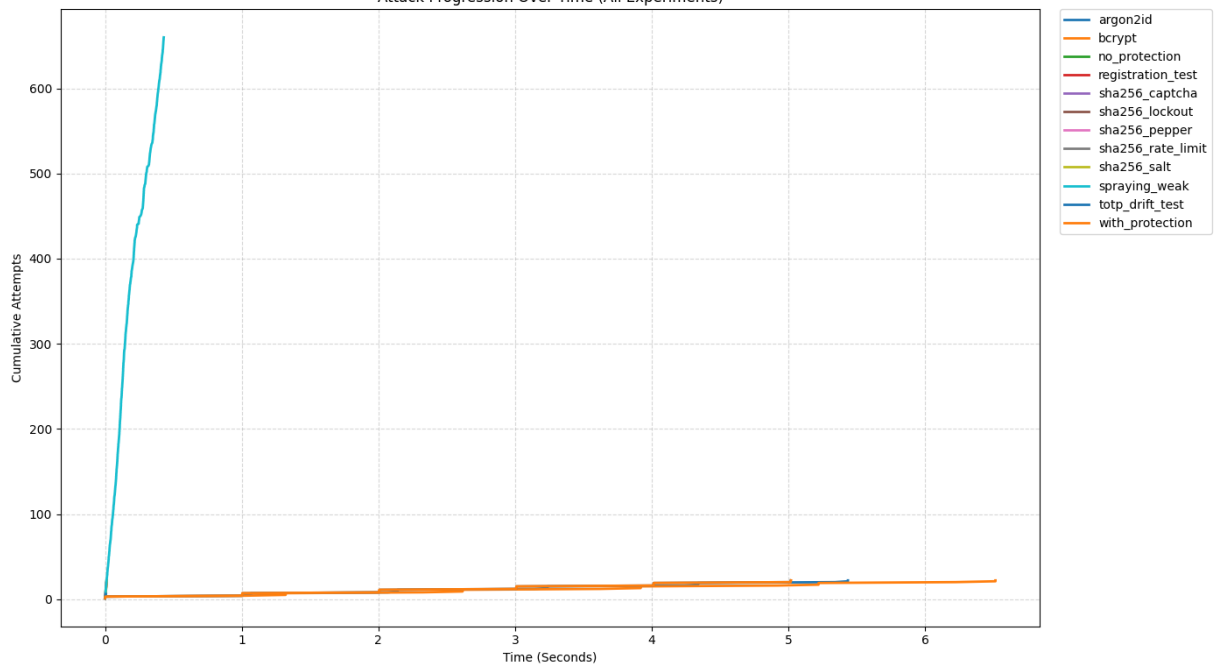
האפקטיביות של CAPTCHA יעילה כשכבת הגנה ראשונה בניסויים שהיא הופעלה בהם, אז המתקפה נחסמה בשלב מוקדם מאוד, עוד לפני שהשרת בכלל בודק את הסיסמה ב DB לכן המסקנה היא שהיא משמשת כמו שומר בכניסה שמבדל ביעילות בין פעולות אוטומטיות שיכולות להיות תוקפים לבין משתמשים אמיתיים אנושיים. בנוסף לשימוש בה יש יתרונות נוספים כמו חיסכון במשאבים, השרת לא מבזבז כוח עיבוד על חישובי גיבוב עבור בוטים כיהם נחסמים כבר בכניסה. בנוסף מניעת נעילות מיותרות במקרה שמשתמש אנושי טעה בהקלדת הסיסמא אז לא יינעל מיד ע"י מנגנוני ההגנה האוטומטים מפני ש CAPTCHA כבר זיהתה שהמשתמש הוא אנושי ולא מתקפה אוטומטית של בוט.

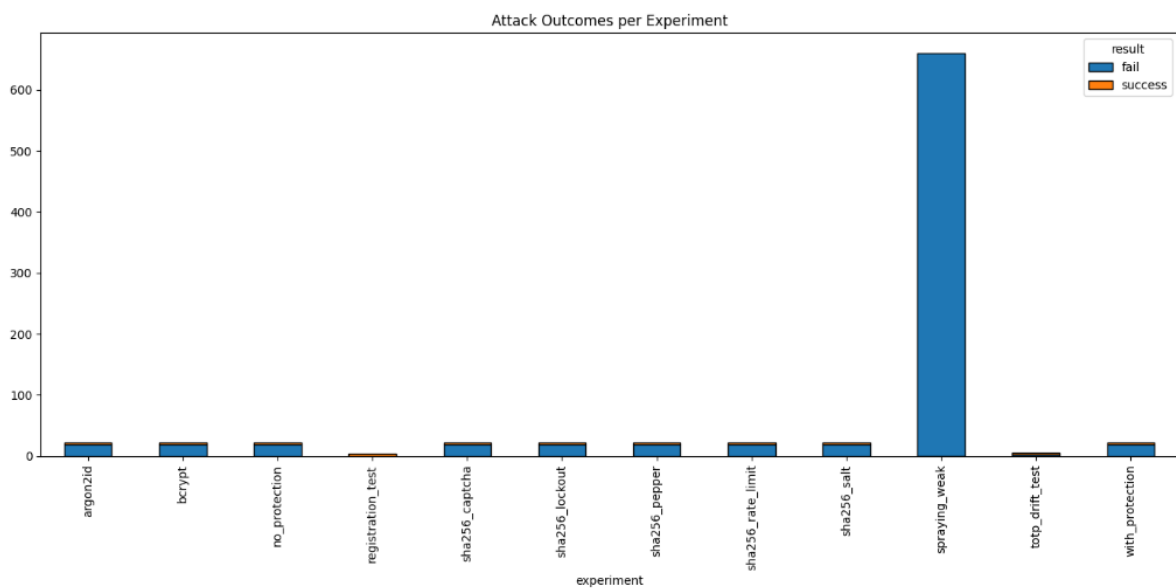
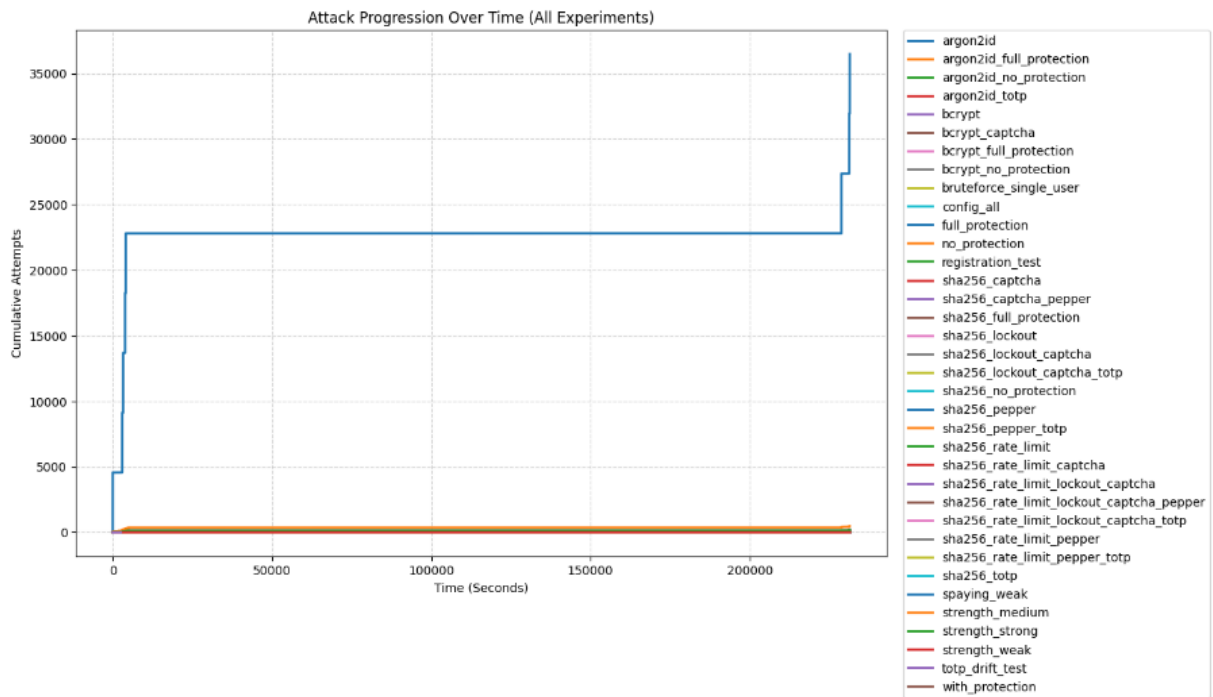


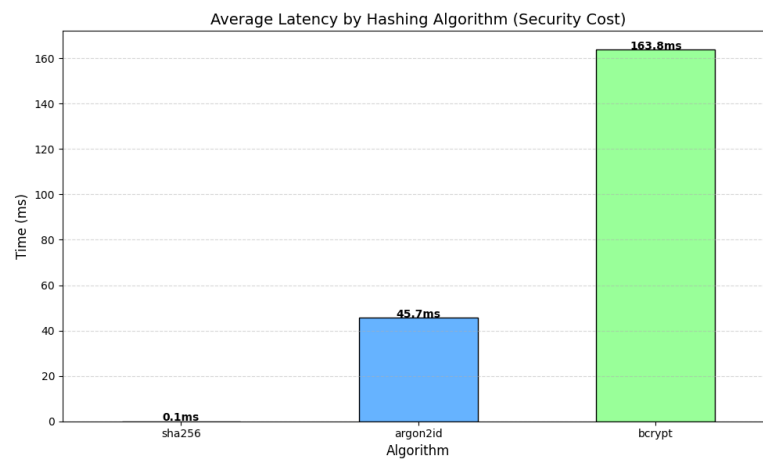
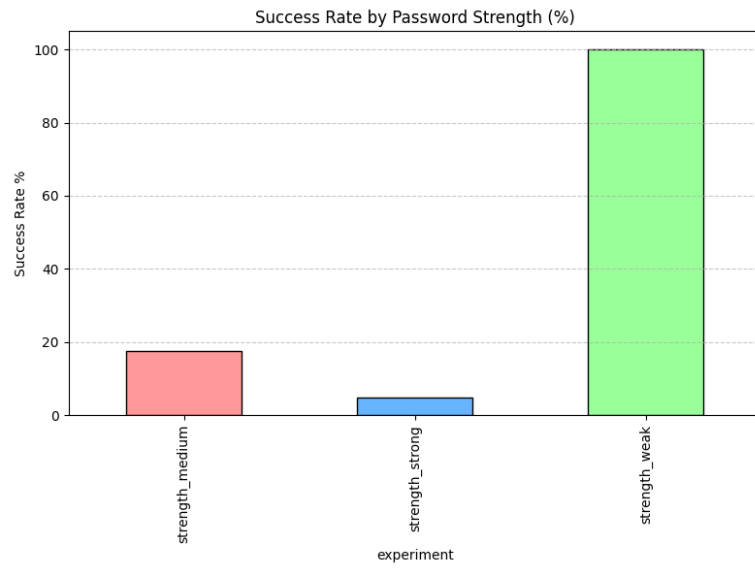
Failure Reasons Analysis



Attack Progression Over Time (All Experiments)







שיקולים אתיים:

במסגרת פרויקט זה מומשו והופעלו כלי תקיפה המדמים התנהגות זדונית. השימוש בכלים אלו מעלה סוגיות אתיות כבדות משקל, שטופלו במסגרת המחקר באופן הבא:

1. כל הניסויים בוצעו בסביבה Sandbox של בדיקות מקומית ומבודדת ללא גישה לרשת החיצונית וללא גישה למערכות אחרות.
2. ה DB נוצר באופן מלאכותי ולא היה שימוש בפרטי משתמשים או סיסמאות אמיתיות.
3. הקוד פותח ומומש אך ורק למטרות של מחקר אקדמי ולמידה של מנגנוני ההגנה באמצעות הקפדה על עקרונות האתיקה המקצועית של אבטחת המידע.
4. על מנת להבטיח את עקביות המחקר בפרויקט, הוגדר נתון קבוע ת.ז (group_seed: 123456789) על מנת להבטיח שיצירת המשתמשים ותרחישי התקיפה אינם רנדומליים אלא עקביים וגם מאפשר בקרה מלאה על הניסוי ומוכיח כי התוצאות שהתקבלו ואחוזי ההצלחה של התוקף הן עקביות.

מקורות:

Standards & Guidelines

RFC 6238: TOTP: Time-Based One-Time

<https://datatracker.ietf.org/doc/html/rfc6238>

OWASP Password Storage Cheat Sheet

הנחיות לשמירה מאובטחת של סיסמאות.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Authentication Cheat Sheet

עקרונות לאימות זהות מאובטח.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Top 10

רשימת סיכוני האבטחה הנפוצים ביותר.

<https://owasp.org/www-project-top-ten/>

NIST Cybersecurity Framework

מסגרת עבודה לניהול סיכוני סייבר.

<https://www.nist.gov/cyberframework>

ספריות פיתוח:

Passlib (bcrypt)

תיעוד ספריית passlib למימוש אלגוריתם bcrypt

<https://passlib.readthedocs.io/en/stable/lib/passlib.hash.bcrypt.html>

Argon2-cffi

תיעוד המימוש של אלגוריתם Argon2 לפיתון.

<https://argon2-cffi.readthedocs.io/en/stable/>

PyOTP

ספרייה ליצירת קודי TOTP ואימות דו-שלבי.

<https://pyauth.github.io/pyotp/>

Python Standard Library

תיעוד רשמי עבור hashlib – sha256 כדי ליצור salt, json, time

<https://docs.python.org/3/library/>

Attack Vectors & Datasets

OWASP Brute Force Attack

הסבר טכני על מתקפות brute force

https://owasp.org/www-community/attacks/Brute_force_attack

OWASP Password Spraying

הסבר טכני על מתקפות על סיסמאות.

https://owasp.org/www-community/attacks/Password_Spraying_Attack

RockYou.txt

מאגר סיסמאות נפוצות שדלפו (שימש לסימולציית התקיפה).

<https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt>

SecLists

אוסף מילונים ורשימות לתקיפות סייבר

<https://github.com/danielmiessler/SecLists>

ניתוח נתונים:

Pandas

ספרייה לניתוח ועיבוד נתונים.

<https://pandas.pydata.org/docs/>

Matplotlib

ספרייה ליצירת גרפים.

<https://matplotlib.org/stable/tutorials/index.html>